ID PPROJECT REPORT

ON

# Get Hired

## Connecting to the work space that you deserve.

## (AI-integrated Full-stack Application)

Submitted

By


| | |
|---|---|
| 231FA04925 | P.Hemanjaneyulu |
| 231FA04974 | G. Somanadh |
| 231FA04421 | P.VarunSai |

*Under the guidance of*

**Department of Computer Science & Engineering**

**SCHOOL OF COMPUTING & INFORMATICS**

**DEPARTMENT OF COMPUTER SCIENCE &ENGINEERING**

**VIGNAN'S FOUNDATION FOR SCIENCE, TECHNOLOGY AND RESEARCH**

**Deemed to be UNIVERSITY**

**Vadlamudi, Guntur.**

**ANDHRA PRADESH, INDIA, PIN-522213**

VIGNAN'S

FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH

(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

# CERTIFICATE

This is to certify that the IDP Project entitled **"Get Hired"** that is being submitted by 231FA04925 (P. Hemajaneyulu), 231FA04974 (G. Somanadh), 231FA04421 (P. Varun Sai) in partial fulfilment of the requirements for the IDP Project, as a part of the academic curriculum of the B.Tech (CSE) Program, Department of Computer Science and Engineering at VFSTR Deemed to be University.

Project Guide

Project Review Committee

HOD, CSE 02/05/26

VIGNAN'S

FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH

(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

# DECLARATION

We hereby declare that the IDP Project entitled " **Get Hired** " that is being submitted by **231FA04925 (P. Hemajaneyulu), 231FA04974 (G. Somanadh), 231FA04421 (P. Varun Sai)** in partial fulfilment of IDP Project course work. This is our original work, and it has not been submitted previously for the award of any degree or diploma. The project has been completed under the supervision of the Department of Computer Science and Engineering, VFSTR Deemed to be University.

By

**231FA04925 (P. Hemajaneyulu),**

**231FA04974 (G. Somanadh),**

**231FA04421 (P. Varun Sai)**

# TABLE OF CONTENTS

LIST OF FIGURES .....7
LIST OF TABLES .....8
ABBREVIATIONS .....9
1. INTRODUCTION .....13
1.1 Introduction .....13
1.2 Project Background .....14
1.3 Objectives .....14
1.4 Project Description .....15
2. SOFTWARE REQUIREMENTS SPECIFICATION (SRS) .....17
2.1 Requirement Analysis .....17
2.2 Problem Statement .....18
2.3 Functional Requirements .....19
2.4 System Features Overview .....20
2.5 Software Requirements .....21
2.6 Hardware Requirements .....22
2.7 Functional Modules .....23
2.8 Non-Functional Requirements .....25
2.9 External Interface Requirements .....26
2.10 Feasibility Study .....27
3. SYSTEM ANALYSIS & DESIGN .....29
3.1 Introduction .....29
3.2 System Overview ..... 30
3.3 System Architecture (GetHired) ..... 32
3.4 Database Design (ER Diagram) .....36
4. SYSTEM MODELING .....40
4.1 UML Diagrams Use Case, Sequence, Activity) .....40
5. IMPLEMENTATION .....48
5.1 Key API Implementation (Flask + REST) .....48
5.2 User Interface Screens (React Frontend) .....56

6. TESTING ..... 64
6.1 Software Testing Overview ..... 64
6.2 Black Box Testing ..... 65
6.3 White Box Testing ..... 66
6.4 Performance Testing ..... 67
6.5 Load Testing ..... 68
6.6 Manual Testing ..... 69
7. RESULTS AND CHALLENGES ..... 71
7.1 Results ..... 71
7.2 Challenges Faced ..... 72
8. CONCLUSION ..... 74
8.1 Conclusion ..... 74
8.2 Scope for Future Work ..... 74
8.3 Limitations ..... 75
BIBLIOGRAPHY ..... 76

# LIST OF FIGURES

**Figure 3-1** System Architecture Diagram (GetHired Platform) ..... 32
**Figure 3-2** Web Application Architecture (Frontend–Backend Flow) ..... 34
**Figure 3-3** Three-Tier Architecture (React + Flask + Database) .....36

**Figure 4-1** Use Case Diagram (GetHired System) ..... 40
**Figure 4-2** Sequence Diagram – Job Application Process ..... 42
**Figure 4-3** Sequence Diagram – AI Resume Scoring ..... 43
**Figure 4-4** Activity Diagram – Job Search & Apply ..... 44
**Figure 4-5** Class Diagram (Core Entities: Users, Jobs, Applications) ..... 45
**Figure 4-6** Deployment Diagram (Production Environment) ..... 47
**Figure 4-7** ER Diagram (Database Relationships) .....48

**Figure 5-1** Home Page (User Dashboard) ..... 55
**Figure 5-2** User Login Interface ..... 56
**Figure 5-3** Job Listing Page ..... 57
**Figure 5-4** Job Details Page ..... 58
**Figure 5-5** Application Submission Interface ..... 59
**Figure 5-6** Worker Profile & Resume Upload ..... 60
**Figure 5-7** Employer Dashboard ..... 61
**Figure 5-8** AI Resume Scoring Result Screen ..... 62
**Figure 5-9** AI Job Match Analysis Interface ..... 63
**Figure 5-10** Cover Letter Generator Output ..... 64

**Figure 6-1** Test Case – Job Creation ..... 66
**Figure 6-2** Test Case – Application Submission ..... 67
**Figure 6-3** API Testing Results (Postman/Response Validation) .....68

# LIST OF TABLES

Table 2.1 User Requirements (Workers, Employers, Representatives) ..... 20

Table 2.2 System Requirements (Software & Hardware) ..... 21

Table 3.1 Database Overview (PostgreSQL / SQLite) ..... 33

Table 3.2 List of Core Database Tables ..... …..34

Table 3.3 Structure of Users Table .....35

Table 3.4 Structure of Jobs Table ..... 36

Table 3.5 Structure of Applications Table ..... 37

Table 3.6 Structure of AI Scores / Feedback Table ..... 38

# ABBREVIATIONS

- **API** : Application Programming Interface
- **AI** : Artificial Intelligence
- **JWT** : JSON Web Token
- **UI** : User Interface
- **UX** : User Experience
- **JSON** : JavaScript Object Notation
- **REST** : Representational State Transfer
- **HTTP** : HyperText Transfer Protocol
- **HTTPS** : HyperText Transfer Protocol Secure
- **CRUD** : Create, Read, Update, Delete
- **DB** : Database
- **SQL** : Structured Query Language
- **ORM** : Object Relational Mapping
- **RDBMS** : Relational Database Management System
- **IDE** : Integrated Development Environment
- **SDK** : Software Development Kit
- **CLI** : Command Line Interface
- **HTML** : HyperText Markup Language
- **CSS** : Cascading Style Sheets
- **JS** : JavaScript
- **TS** : TypeScript
- **RBAC** : Role-Based Access Control
- **MVC** : Model-View-Controller
- **SRS** : Software Requirements Specification

# CHAPTER - 1

# INTRODUCTION

# 1. INTRODUCTION

## 1.1 Introduction

The GetHired project focuses on the development of a modern web-based recruitment platform that enables users to search, apply, and manage job opportunities efficiently from anywhere at any time. The primary goal of this system is to streamline the hiring process by connecting job seekers (workers) with employers through a centralized and intelligent platform.

Traditional job application processes are often time-consuming, inefficient, and lack personalization. GetHired addresses these challenges by providing a digital solution that simplifies job discovery, application submission, and candidate evaluation. The platform integrates advanced features such as AI-powered resume analysis, job matching, and automated feedback, making the hiring process faster and more effective.

Through this system, users can browse job listings, filter opportunities based on their preferences, and apply directly through the platform. Employers can post job openings, review applications, and manage candidates efficiently. Additionally, the system provides AI-driven insights to help both employers and applicants make informed decisions.

The platform supports multiple user roles, ensuring tailored functionality for each type of user:

1. Worker (Job Seeker):

Workers can register and log in to the platform, create and update their profiles, upload resumes, and apply for jobs. They also benefit from AI-based features such as resume scoring, job recommendations, and interview preparation assistance.

2. Employer:

Employers can create and manage job postings, view applications submitted by workers, and evaluate candidates using AI-generated scores and feedback. This helps in selecting the most suitable candidates efficiently.

3. Representative (Admin):

Representatives act as administrators who manage users, monitor platform activity, resolve disputes, and ensure smooth system operation.

The system is designed to be scalable and can be deployed on cloud platforms such as AWS or other hosting environments, ensuring reliability and accessibility.

## 1.2 Project Background

In today's digital era, recruitment processes are rapidly evolving, yet many traditional hiring systems still suffer from inefficiencies such as manual screening, lack of personalization, and

delayed communication between employers and job seekers. These challenges often result in increased hiring time, poor candidate-job matching, and limited accessibility.

With the rise of cloud computing and artificial intelligence, there is a growing need to modernize recruitment systems. The GetHired platform is developed to address these issues by providing an intelligent, automated, and user-friendly solution. By integrating AI-driven analysis and cloud-based accessibility, the system enhances the efficiency of hiring workflows and improves the overall experience for both employers and job seekers.

The project aims to transform the conventional recruitment process into a smart, data-driven system that enables faster decision-making, better candidate evaluation, and seamless interaction between users.

## 1.3 Objective

The primary objective of the GetHired system is to **automate and optimize the recruitment process** using modern web technologies and artificial intelligence.

The key objectives include:

- To provide a centralized platform for job seekers and employers
- To enable users to search and apply for jobs anytime and from anywhere
- To assist employers in efficiently managing job postings and applications
- To implement AI-based features such as resume scoring, job matching, and feedback generation
- To improve decision-making through data-driven insights
- To reduce manual effort and time involved in recruitment processes
- To ensure secure authentication and role-based access control

Overall, the system aims to create a **smart, scalable, and efficient hiring ecosystem**.

## 1.4 Project Description

This section provides an overview of the main functionalities of the GetHired application.

The system supports three primary user roles:

**1. Worker (Job Seeker):**

- Sign up and log in to the platform
- Create and manage personal profile

- Upload resume and receive AI-based scoring
- Search and filter job opportunities
- Apply for jobs
- View application status and feedback
- Receive job recommendations

**2. Employer:**

- Register and log in to the system
- Create, update, and manage job postings
- View and manage job applications
- Analyze candidates using AI-generated scores and feedback
- Track hiring progress through dashboards

**3. Representative (Admin):**

- Manage users (workers and employers)
- Monitor platform activities
- Handle disputes and reports
- View analytics and system insights
- Ensure system integrity and smooth operation

**Employer Functionalities:**

After logging into the system, the employer has access to the following features:

- **Create Job Listings**
- **View Applications**
- **Update Application Status**
- **Logout from Account**

**a) Sign Up:**

A new employer can register on the platform by providing required details such as:

- Company name
- Email address
- Password
- Industry and organization details

After successful registration, the employer account is created and ready for use.

**b) Login:**

The employer logs into the system using registered email and password credentials.
If the employer forgets the password, they can use the **"Forgot Password"** option to reset it via email.

**i. Create Job Listings:**

After logging in, the employer can:

- Add job title, description, and requirements
- Specify job location, salary range, and job type
- Publish job postings

**ii. View Applications:**

Employers can view all applications submitted by workers for their job postings, including:

- Candidate details
- Resume information
- AI-generated scores

**iii. Update Application Status:**

Employers can update the status of applications based on evaluation:

- Accepted
- Rejected
- Under Review

# CHAPTER – 2

# SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

# 2. SOFTWARE REQUIREMENTS SPECIFICATION

## 2.1 Requirement Analysis

To ensure accessibility, scalability, and ease of use, the GetHired system is designed as a **web-based application** that can be accessed from any device with an internet connection. The platform is developed using modern technologies and is compatible with operating systems such as Windows, Linux, and macOS.

The system is structured to provide seamless interaction between users and the backend services through RESTful APIs. It enables efficient data processing, storage, and retrieval.

The requirement analysis phase includes the preparation of the following documents and models:

1. **Problem Statement**
2. **Data Flow Diagrams (DFD)**
3. **Use Case Diagrams**
4. **Other UML Diagrams (Sequence, Activity, Class)**

These artifacts provide a clear and structured visualization of the system's functionality and workflow.

## 2.2 Problem Statement

The traditional recruitment process is often inefficient, involving manual resume screening, lack of proper job-candidate matching, and delayed communication between employers and job seekers. These limitations lead to increased hiring time and reduced effectiveness in candidate selection.

The problem addressed by the GetHired system is to develop a **centralized and intelligent recruitment platform** that:

- Stores and manages user and job data efficiently
- Automates application processing
- Provides AI-based resume scoring and job matching
- Enhances communication between employers and workers

The system ensures secure data handling and efficient interaction between the frontend and backend services.

## 2.3 Functional Requirements

**1. User Registration (Worker)**

1.1 The system shall allow new users (workers) to register using:

- Name
- Email address
- Password

1.2 The system shall validate user input and ensure unique email registration.

1.3 The system shall allow users to log in using their credentials.

1.4 The system shall provide password recovery functionality.

1.5 The system shall allow users to update their profile information, including:

- Skills
- Experience level
- Resume upload

**2. User Registration (Employer)**

2.1 The system shall allow employers to register by providing:

- Company name
- Email address
- Password

2.2 The system shall validate employer credentials and create a secure account.

2.3 The system shall allow employers to log in and manage their profiles.

**3. Job Management**

3.1 Employers shall be able to create job postings.

3.2 Employers shall be able to update and delete job listings.

3.3 The system shall allow users to view available jobs with filtering options (location, title, salary, etc.).

**4. Application Management**

4.1 Workers shall be able to apply for jobs.

4.2 The system shall store application details securely.

4.3 Employers shall be able to view applications submitted for their jobs.

4.4 Employers shall be able to update application status (accepted, rejected, reviewing).

### 5. AI Features

5.1 The system shall provide resume scoring based on job requirements.

5.2 The system shall analyze job-candidate matching and provide recommendations.

5.3 The system shall generate AI-based feedback for applications.

## 2.4 Software Requirement Specification

The GetHired system is developed using modern web technologies to ensure scalability, performance, and maintainability. The backend is implemented using Python (Flask Framework), while the frontend is built using React (TypeScript) for a dynamic and responsive user interface.

The system follows a RESTful API architecture, enabling smooth communication between frontend and backend components. Data is stored using SQLite/PostgreSQL databases, and optional caching can be implemented using Redis for performance optimization.

The platform also integrates external AI services (such as Gemini or Groq APIs) to provide intelligent features like resume scoring, job matching, and feedback generation.

The system can be deployed on cloud platforms such as AWS, Heroku, or similar environments, ensuring high availability and scalability.

### 2.4.1 Purpose

The purpose of this document is to provide a detailed description of the GetHired recruitment system. It outlines the system's functionality, features, constraints, and operational environment.

This document serves as a reference for:

Developers implementing the system

Stakeholders understanding system behavior

Testers validating system requirements

It also describes both functional and non-functional requirements, ensuring that the system meets performance, security, and usability standards.

### 2.4.2 Scope of the Project

The GetHired system is a web-based recruitment platform designed to connect job seekers and employers through a centralized system. The platform supports multiple user roles and provides AI-driven features to enhance hiring efficiency.

1. Employers:

Employers can:

Register and log in to the system

Create, update, and manage job postings

View applications submitted by workers

Evaluate candidates using AI-generated scores and feedback

Update application status (accepted, rejected, reviewing)

2. Workers (Job Seekers):

Workers can:

Register and log in to the platform

Create and manage their profiles

Upload resumes

Search and filter job opportunities

Apply for jobs

View application status and history

Receive AI-based job recommendations and feedback

3. Representatives (Admin):

Representatives can:

Manage users and monitor platform activity

Handle disputes and reports

Access analytics and system insights

Ensure system security and proper functioning

2.4.3 Technologies Used

1. Backend – Flask (Python)

Flask is a lightweight and flexible web framework used to build RESTful APIs. It handles request processing, business logic, and integration with databases and AI services.

2. Frontend – React (TypeScript)

React is used to build a responsive and interactive user interface. TypeScript ensures type safety and improves code maintainability.

3. Database – SQLite / PostgreSQL

The system uses relational databases for storing structured data such as users, jobs, and applications.

4. AI Integration (Gemini / Groq APIs)

AI services are integrated to provide advanced features such as:

Resume scoring

Job matching analysis

Feedback generation

5. API Communication – REST (JSON)

The system uses REST APIs with JSON format for communication between frontend and backend.

6. Authentication – JWT (JSON Web Token)

JWT is used to securely authenticate users and manage sessions.

7. Deployment Platforms

The system can be deployed using:

AWS (EC2, S3, RDS)

Heroku

Docker containers

## 2.5 Software Requirements:

The GetHired system is developed using modern web technologies that ensure scalability, flexibility, and performance. The software requirements include the operating environment, development tools, and technologies used to build and deploy the application.

Software Specifications:

Operating System : Windows, Linux (Ubuntu), macOS

Programming Language : Python (Backend), TypeScript/JavaScript (Frontend)

Backend Framework : Flask

Frontend Framework : React (with Vite)

API Architecture : RESTful APIs (JSON-based communication)

HTTP Version : HTTP/1.1 / HTTP/2

Database : SQLite (Development) / PostgreSQL (Production)

ORM : SQLAlchemy

Authentication : JWT (JSON Web Token)

AI Integration : Gemini API / Groq API

Cloud Platform : AWS / Heroku / Docker-based deployment

Web Server : Gunicorn / uWSGI (for production)

Version Control : Git (GitHub)

Development Tools: VS Code / PyCharm

**Technologies Used:**

Python, Flask

React, TypeScript

REST APIs

SQL (PostgreSQL/SQLite)

AI APIs (Gemini, Groq)

## 2.6 Hardware Requirements

The GetHired system is a web-based application, meaning it does not require high-end hardware on the client side. However, both client and server environments should meet minimum requirements to ensure smooth performance.

| Component | Requirement |
|---|---|
| Operating System | Windows / Linux / macOS / Android / iOS |
| Software | Modern Web Browser (Chrome, Firefox, Edge) |
| Processor | Dual-Core CPU or higher |
| RAM | Minimum 2 GB (Recommended 4 GB) |
| Disk Space | Minimum 200 MB free |
| Internet | Stable internet connection |

Table 2.1: Client Requirements

Server Requirements:

| Component | Requirement |
|---|---|
| Operating System | Linux (Ubuntu 20.04+ preferred) |
| Software | Python 3.x, Flask, PostgreSQL / SQLite |
| Web Server | Gunicorn / uWSGI |

| | |
|---|---|
| Processor | Multi-Core CPU (2+ cores recommended) |
| RAM | Minimum 4 GB (Recommended 8 GB+) |
| Disk Space | Minimum 1 GB (scalable based on data growth) |
| Cloud Platform | AWS / Heroku / Docker-supported environment |

Table 2.2: Server Requirements

## 2.7 Functional Requirements (Modules):

The GetHired system consists of multiple modules based on user roles. Each module provides specific functionalities tailored to the needs of different users. The main roles in the system are:

- **Worker (Job Seeker)**
- **Employer**
- **Representative (Admin)**

**1. Worker (Job Seeker)**

**i) Sign Up**

A new worker can register on the platform by providing:

- Name
- Email address
- Password

After successful registration, a new user account is created.

**ii) Login**

The worker logs into the system using registered credentials.
If the user forgets the password, they can reset it using the **"Forgot Password"** option.

**iii) Search Jobs**

After logging in, the worker can:

- Browse available job listings
- Filter jobs based on location, title, salary, and job type

**iv) Apply for Job**

- The worker can select a job and submit an application
- Optionally include a cover letter
- The system stores the application and confirms submission

**v) View Application Status**

- The worker can track application status:
    - Pending
    - Reviewing
    - Accepted
    - Rejected

**vi) AI Features**

- Resume scoring
- Job recommendation system
- AI-based feedback on applications
- Interview preparation assistance

**vii) Logout**

The worker can securely log out of the system.

**2. Employer**

**i) Sign Up**

Employers register by providing:

- Company name
- Email
- Password

**ii) Login**

Employers log in using their credentials with password recovery options available.

**iii) Manage Job Listings**

- Create job postings
- Edit job details
- Delete job listings

**iv) View Applications**

- View all applications submitted for jobs
- Access candidate details and resumes

**v) Update Application Status**

- Accept, reject, or mark applications as reviewing
- Notify candidates about status updates

**vi) AI-Based Candidate Evaluation**

- View AI-generated scores
- Analyze candidate-job matching
- Use feedback for better hiring decisions

**vii) Logout**

Employers can log out securely.

**3. Representative (Admin)**

**i) Login**

Admin logs into the system with secure credentials.

**ii) Manage Users**

- View all users
- Ban or activate users
- Monitor user activities

**iii) Manage Applications & Disputes**

- View all applications
- Handle disputes and reports

**iv) View Analytics**

- Access system-wide statistics
- Monitor platform performance

**v) Logout**

Admin can securely log out of the system.

## 2.8 Non-Functional Requirements

Non-functional requirements define the quality attributes of the GetHired system, ensuring performance, usability, reliability, and scalability.

### 2.8.1 Performance Requirements

- The system shall provide fast and responsive user interactions.
- The response time for most API requests should not exceed **2–5 seconds** under normal conditions.
- User authentication (login/verification) should be completed within **a few seconds**.
- The system shall efficiently handle multiple concurrent users.
- AI-based operations (resume scoring, job matching) should return results within an acceptable time frame.

### 2.8.2 Design Constraints

- The system shall be developed as a **web-based application**.
- Backend must be implemented using **Flask (Python)**.
- Frontend must be implemented using **React (TypeScript)**.
- The system shall follow a **RESTful API architecture**.
- The database must use a **relational database system (SQLite/PostgreSQL)**.

### 2.8.3 Standards Compliance

- The system shall follow standard web protocols such as **HTTP/HTTPS**.
- Data exchange shall be in **JSON format**.
- The UI shall maintain consistency in layout, design, and navigation.
- Coding standards and best practices shall be followed for maintainability.

### 2.8.4 Availability

- The system shall be available **24/7** for users with internet access.
- Downtime should be minimized through proper deployment and monitoring.

### 2.8.5 Portability

- The system shall be accessible through any modern web browser.
- It shall support multiple platforms including **desktop, tablet, and mobile devices**.

### 2.8.6 Reliability

- The system shall ensure reliable performance under normal and peak loads.
- Data integrity must be maintained during all operations.
- Error handling mechanisms shall be implemented to prevent system failures.

## 2.9 External Interface Requirements

### 2.9.1 User Interface

The user interface of the GetHired system is designed to be **simple, intuitive, and user-friendly**.

- The application provides a clean dashboard for each user role:
  - Worker Dashboard
  - Employer Dashboard
  - Admin Panel
- Key UI features:
  - Easy navigation between pages
  - Responsive design for all devices
  - Clear forms for job posting and applications
  - Visual feedback for actions (success, error messages)
- The frontend is developed using:
  - **React (TypeScript)**
  - HTML, CSS for layout and styling

### 2.9.2 Database (Data Interface)

The system interacts with a relational database to store and manage data.

**Users Table**

- User ID
- Name
- Email
- Password (hashed)
- Role (Worker / Employer / Admin)

**Jobs Table**

- Job ID

- Title
- Description
- Requirements
- Location
- Salary range
- Employer ID

**Applications Table**

- Application ID
- Job ID
- Worker ID
- Status
- AI Score
- Feedback

**AI Data (Optional Storage)**

- Resume analysis results
- Match scores
- Suggestions

## 2.10 Feasibility Study

The feasibility study evaluates the practicality and viability of the GetHired system in terms of organizational, economic, technical, and behavioral aspects.

### 2.10.1 Organizational Feasibility

The GetHired platform aligns with the objectives of modern recruitment systems by providing a centralized and efficient hiring solution. It enhances the recruitment workflow by reducing manual effort and improving candidate-employer interaction.

The system supports:

Faster hiring processes

Better decision-making using AI insights

Improved user experience for both employers and job seekers

Additionally, the platform is flexible and scalable, allowing organizations to expand features and adapt to future requirements.

2.10.2 Economic Feasibility

The project is economically feasible as it requires minimal investment:

The system is web-based and does not require expensive hardware

Open-source technologies (Flask, React, PostgreSQL) reduce development costs

Cloud deployment options (AWS, Heroku) offer cost-effective scalability

The only recurring costs may include:

Cloud hosting services

Domain and maintenance costs

Optional AI API usage charges

Overall, the system provides a cost-effective solution for modern recruitment needs.

2.10.3 Technical Feasibility

The GetHired system is technically feasible due to the availability of modern tools and technologies:

Backend developed using Flask (Python)

Frontend built with React (TypeScript)

Database managed using SQLite/PostgreSQL

AI integration using external APIs (Gemini, Groq)

The system architecture supports:

Scalability

Secure authentication (JWT)

Efficient API communication

The successful integration of these technologies ensures that the system can be developed, deployed, and maintained effectively.

### 2.10.4 Behavioral Feasibility

The system is designed to be user-friendly and intuitive, requiring minimal technical knowledge from users.

Simple registration and login process

Easy navigation for job search and application

Clear dashboards for employers and administrators

AI features presented in an understandable way

Users can interact with the system comfortably without requiring extensive training, ensuring high user acceptance.

# CHAPTER – 3

# SYSTEM ANALYSIS & DESIGN

## 3. ANALYSIS & DESIGN

### 3.1 Introduction

### 3.1.1 Purpose

The purpose of this section is to describe the overall design and structure of the GetHired system. It provides a clear understanding of how the system is organized, how different components interact, and how the requirements defined earlier are translated into a working solution.

This section acts as a bridge between requirement analysis (Chapter 2) and system implementation (Chapter 5) by defining the architecture, modules, and workflows of the system.

#### 3.1.1.1 Document Purpose

The System Design Document (SDD) presents a detailed representation of the GetHired platform. It serves as a communication medium for developers, designers, and stakeholders to understand the system's structure and behavior.

This document includes:

High-level system architecture

Module design and interactions

Data flow and processing

User interface structure

It helps ensure that all stakeholders have a shared understanding of the system design before implementation begins.

#### 3.1.1.2 Project Purpose

The primary purpose of the GetHired system is to develop a fully functional web-based recruitment platform that connects job seekers and employers through a centralized system.

The system enables:

Efficient job posting and management

Seamless application submission and tracking

AI-driven resume analysis and job matching

Real-time interaction between users

The platform is designed to operate over the internet, making it accessible from anywhere, and aims to replace traditional, manual recruitment processes with an intelligent and automated solution.

### 3.1.2 Scope

This section defines the scope of the system design and outlines the boundaries of the GetHired platform.

#### 3.1.2.1 Document Scope

This document provides a high-level overview of the system architecture and design. It focuses on:

System components and their interactions

User interface design

Data flow between modules

Overall structure of the application

However, the document does not include:

Low-level implementation details

Detailed code-level descriptions

Internal logic of every function

The goal is to present a clear architectural view that guides development without going into unnecessary technical depth.

#### 3.1.2.2 Project Scope

The GetHired system consists of multiple modules designed to support different user roles and functionalities:

Worker Module → Job search, application submission, AI features

Employer Module → Job posting, application management, candidate evaluation

Admin (Representative) Module → User management, analytics, system monitoring

The system also includes:

RESTful API integration

Database management system

AI-based services for resume scoring and job matching

The scope of the project is limited to:

Web-based application functionality

Core recruitment features

AI-assisted decision-making tools

Future enhancements may include mobile applications, advanced analytics, and real-time communication features.

## 3.2 System Overview

The GetHired system is a web-based recruitment platform designed to provide a seamless interaction between job seekers and employers. It is built using a modern technology stack and follows a three-tier architecture, ensuring scalability, performance, and maintainability.

The system operates through a combination of frontend interfaces, backend services, and database management, integrated with AI services to enhance functionality.

### 3.2.1 Development Tools & Technologies

The development of the GetHired platform utilizes the following tools and technologies:

#### 3.2.1.1 Visual Studio Code

Visual Studio Code is used as the primary development environment for both frontend and backend development. It is a lightweight and powerful code editor that supports multiple programming languages such as Python, JavaScript, and TypeScript.

It provides:

Syntax highlighting and debugging tools

Extensions for React, Flask, and API testing

Integrated terminal for development and deployment

#### 3.2.1.2 Python (Flask Framework)

Python is used as the backend programming language, and Flask is used as the web framework to build RESTful APIs.

Flask provides:

Lightweight and flexible backend development

Easy integration with databases and AI services

Efficient handling of HTTP requests and response.

3.2.1.3 React (Frontend Framework)

React is used to develop the user interface of the system. It allows the creation of dynamic, responsive, and component-based web applications.

Key features:

Component-based architecture

Fast rendering using Virtual DOM

Seamless API integration

3.2.1.4 Database (SQLite / PostgreSQL)

The system uses relational databases to store structured data such as users, jobs, and applications.

SQLite is used for development

PostgreSQL is used for production

The database ensures:

Data integrity

Efficient querying

Scalability

3.2.1.5 API Communication (REST + JSON)

The frontend and backend communicate through RESTful APIs using JSON format.

This ensures:

Loose coupling between frontend and backend

Scalability and flexibility

Easy integration with external services

3.2.1.6 AI Integration (Gemini / Groq APIs)

AI services are integrated into the system to enhance functionality:

Resume scoring

Job matching analysis

Feedback generation

Interview preparation

These features improve decision-making and user experience.

#### 3.2.1.7 Version Control (Git & GitHub)

Git is used for version control, enabling:

Code tracking

Collaboration

Version management

GitHub is used as the repository hosting platform.

```
gethired/
│
├── backend/
│   ├── app.py
│   ├── config.py
│   ├── database.py
│   │
│   ├── models/
│   │   ├── user_model.py
│   │   ├── job_model.py
│   │   ├── application_model.py
│   │   └── ...
.   .
.   .
```

**Figure 3-1:** Directory structure of the GetHired system showing separation between frontend, backend, and documentation layers.

## 3.3 System Architecture

### 3.3.1 Architectural Design

The GetHired platform follows a modern web application architecture that connects multiple components to deliver a seamless and efficient user experience. It enables communication between users and the system through a browser-based interface, where requests are processed by the backend and responses are returned dynamically.

The architecture is designed to be:

Scalable

Secure

Maintainable

Performance-efficient

The system follows a three-tier architecture model, consisting of:

Presentation Layer (Frontend – React)

Application Layer (Backend – Flask API)

Data Layer (Database + AI Services)

Attributes of a Well-Built Web Application Architecture

Solves real-world recruitment problems efficiently

Provides a clean and user-friendly interface

Ensures fast response and smooth user experience

Supports analytics and monitoring

Maintains high security (authentication & authorization)

Scales effectively with increasing users and data

Enables easy debugging and logging

Supports automation through AI integration

Components of GetHired Architecture

The architecture of the GetHired system consists of multiple components that work together:

1. UI/UX Components (Frontend Layer)

These components are responsible for user interaction and visual representation.

They include:

User dashboards (Worker, Employer, Admin)

Job listing and search pages

Application forms

Profile management interfaces

Notifications and alerts

The frontend is built using:

React (TypeScript)

HTML, CSS

These components ensure a responsive and user-friendly experience across devices.

2. Structural Components (Backend & Data Layer)

These components handle the core functionality and system operations.

a) Backend Server (Flask API)

Processes incoming requests

Implements business logic

Handles authentication (JWT)

Manages communication with database and AI services

b) Database Layer

Stores structured data:

Users

Jobs

Applications

Ensures data integrity and efficient querying

Uses SQLite (development) / PostgreSQL (production)

c) AI Services Layer

Provides intelligent features:

Resume scoring

Job matching

Feedback generation

Integrates with external APIs (Gemini, Groq)

d) API Communication Layer

Uses RESTful APIs

Exchanges data in JSON format

Connects frontend and backend seamlessly

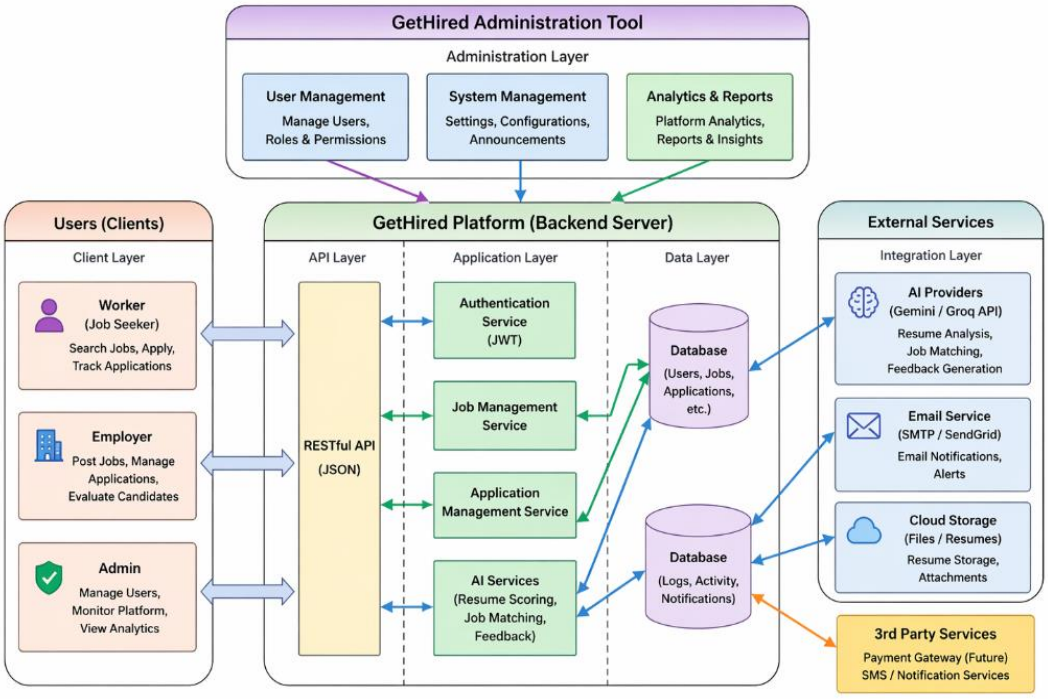



**Figure 3-2 GetHired Web Application Architecture Diagram**

## 3.3.2 Build and Deploy to Web Application in Cloud

The deployment process of the GetHired system involves building both the backend and frontend applications and deploying them to a cloud platform such as AWS, Heroku, or Docker-based environments.

The system is designed to be cloud-ready, ensuring scalability, high availability, and easy access for users from anywhere.

Deployment Overview

The deployment process includes the following steps:

Backend Preparation (Flask API):

Configure environment variables

Install dependencies using requirements.txt

Set up database (SQLite/PostgreSQL)

Configure JWT authentication

Frontend Build (React Application):

Install dependencies using npm install

Build production-ready files using:

```
npm run build
```

This generates optimized static files in the dist/ or build/ directory.

Server Configuration:

Use Gunicorn or uWSGI as the production server

Configure environment variables (API URLs, database credentials)

Enable CORS for frontend-backend communication

Cloud Deployment Options:

a) AWS Deployment

Use EC2 for backend hosting

Use S3 + CloudFront for frontend hosting

Use RDS (PostgreSQL) for database

Configure security groups and load balancer

b) Heroku Deployment

Deploy backend using Git

Configure environment variables

Use Heroku Postgres for database

Deploy frontend separately or integrate with backend

c) Docker Deployment

Containerize backend and frontend

Use Docker Compose for multi-service setup

Ensure portability across environments

5. Running the Application

After deployment:

The backend API is accessible via a public URL

The frontend is hosted and connected to the API

Users can access the platform through a web browser

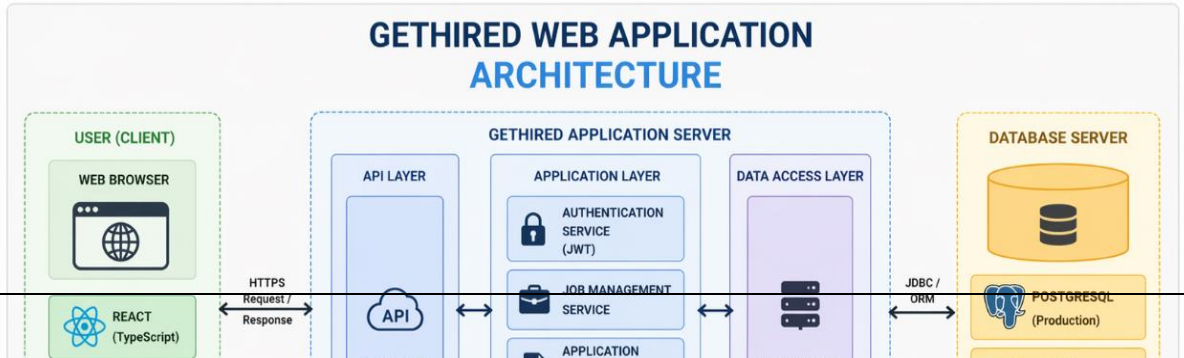
6. Monitoring & Logs

Logs can be monitored using:

Cloud provider dashboards (AWS CloudWatch, Heroku logs)

Errors and performance issues can be tracked and resolved efficiently

**3.3.4 Overall Software Architecture**



## 3.4 Data Design

### 3.4.1 Database

The GetHired system uses a Relational Database Management System (RDBMS) to store and manage structured data efficiently.

Development Database: SQLite

Production Database: PostgreSQL

The database is designed to ensure:

Data integrity

Efficient querying

Scalability for large datasets

Table 3.1: Database Name

| **Name** |
| --- |
| gethired_db |

**3.4.2 Tables**

The database consists of multiple tables that store different types of data required for the system.

**Table 3.2: List of Database Tables**

| Name | Description |
|---|---|
| **Users** | **Stores user details (workers, employers, admin)** |
| **Jobs** | **Stores job postings** |
| **Applications** | **Stores job application details** |
| **AI_Analysis** | **Stores AI-generated scores and feedback** |

### 3.4.2.1 Table: Users

**Description**

Stores information about all users in the system, including workers, employers, and administrators.

| Name | Type | Null | Default | Index |
|---|---|---|---|---|
| user_id | Integer | No | Auto | Primary Key |
| name | Varchar(100) | No | - | - |
| email | Varchar(100) | No | - | Unique |
| password | Varchar(255) | No | - | - |
| role | Varchar(20) | No | "worker" | - |
| created_at | Timestamp | Yes | Current | - |

**Definition:**

```
CREATE TABLE Users (
    user_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    role VARCHAR(20) DEFAULT 'worker',
```

```
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

#### 3.4.2.2 Table: Jobs

**Description**

Stores job postings created by employers.

| Name | Type | Null | Default | Index |
|---|---|---|---|---|
| job_id | Integer | No | Auto | Primary Key |
| title | Varchar(150) | No | - | - |
| description | Text | No | - | - |
| location | Varchar(100) | Yes | NULL | - |
| salary | Varchar(50) | Yes | NULL | - |
| employer_id | Integer | No | - | Foreign Key |
| created_at | Timestamp | Yes | Current | - |

**Definition:**

```
CREATE TABLE Jobs (
    job_id SERIAL PRIMARY KEY,
    title VARCHAR(150) NOT NULL,
    description TEXT NOT NULL,
    location VARCHAR(100),
    salary VARCHAR(50),
    employer_id INTEGER REFERENCES Users(user_id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

#### 3.4.2.3 Table: Applications

**Description**

Stores job applications submitted by workers.

| Name | Type | Null | Default | Index |
|---|---|---|---|---|
| application_id | Integer | No | Auto | Primary Key |
| job_id | Integer | No | - | Foreign Key |
| user_id | Integer | No | - | Foreign Key |
| status | Varchar(20) | Yes | "pending" | - |
| applied_at | Timestamp | Yes | Current | - |

### 3.4.2.3 Table: Applications

**Description**

Stores job applications submitted by workers.

| Name | Type | Null | Default | Index |
|---|---|---|---|---|
| application_id | Integer | No | Auto | Primary Key |
| job_id | Integer | No | - | Foreign Key |
| user_id | Integer | No | - | Foreign Key |
| status | Varchar(20) | Yes | "pending" | - |
| applied_at | Timestamp | Yes | Current | - |

**Definition:**

```
CREATE TABLE Applications (
    application_id SERIAL PRIMARY KEY,
    job_id INTEGER REFERENCES Jobs(job_id),
    user_id INTEGER REFERENCES Users(user_id),
    status VARCHAR(20) DEFAULT 'pending',
    applied_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);.
```

#### 3.4.2.4 Table: AI_Analysis

**Description**

Stores AI-generated analysis results such as resume scores and feedback.

| Name | Type | Null | Default | Index |
|---|---|---|---|---|
| analysis_id | Integer | No | Auto | Primary Key |
| application_id | Integer | No | - | Foreign Key |
| score | Float | Yes | NULL | - |
| feedback | Text | Yes | NULL | - |
| created_at | Timestamp | Yes | Current | - |

**Definition:**

```
CREATE TABLE AI_Analysis (
    analysis_id SERIAL PRIMARY KEY,
    application_id INTEGER REFERENCES Applications(application_id),
    score FLOAT,
    feedback TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

# CHAPTER – 4

# MODELING

## 4. MODELING

## 4.1 Design

The design phase of the GetHired system is based on a structured approach that follows **Object-Oriented Analysis and Design (OOAD)** principles. After gathering and analyzing system requirements, the system is modeled using **Unified Modeling Language (UML)** diagrams.

UML diagrams help in:

- Visualizing system functionality
- Understanding interactions between users and the system
- Designing system structure before implementation
- Documenting system behavior in a clear and standardized way

The modeling phase ensures that the system is well-organized, scalable, and easy to maintain.

### 4.1.1 Use Case Diagram

In Unified Modeling Language (UML), a **Use Case Diagram** represents the functional requirements of the system by illustrating interactions between users (actors) and the system.

Actors represent different types of users who interact with the system. In the GetHired platform, the main actors are:

- **Worker (Job Seeker)**
- **Employer**
- **Admin (Representative)**

Each actor performs specific actions (use cases) within the system.

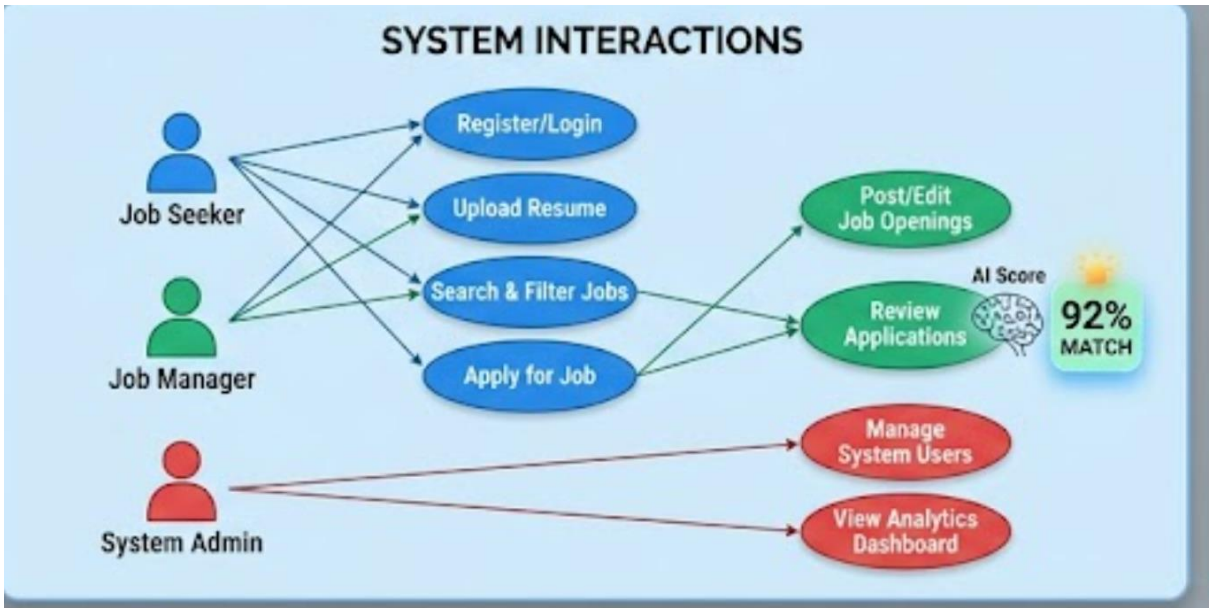



**Figure 4-1:** Use Case Diagram representing interactions between Worker, Employer, Admin, and the GetHired system.

### 4.1.2 Sequence Diagram

A **Sequence Diagram** in UML is used to represent how different components of a system interact with each other in a specific sequence over time. It shows the flow of messages between objects and how operations are carried out step-by-step.

In the GetHired system, sequence diagrams help in understanding how users interact with the system and how the backend processes requests and returns responses.

A sequence diagram typically includes:

- **Actors (Users)**
- **Lifelines (System components)**
- **Messages (Requests & responses)**
- **Execution flow (top to bottom)**

**Sequence Diagram – Job Application Process (Worker Module)**

The following sequence describes how a worker applies for a job in the GetHired system:

1. The **Worker** logs into the system using valid credentials
2. The request is sent to the **Frontend (React UI)**
3. The frontend sends an API request to the **Backend (Flask Server)**
4. The backend verifies authentication and retrieves job listings from the **Database**
5. The worker selects a job and clicks **Apply**
6. The frontend sends the application request to the backend
7. The backend stores the application in the **Database**
8. The backend triggers **AI Services** to analyze the resume and generate a score
9. The system returns confirmation and feedback to the frontend
10. The worker can view the application status

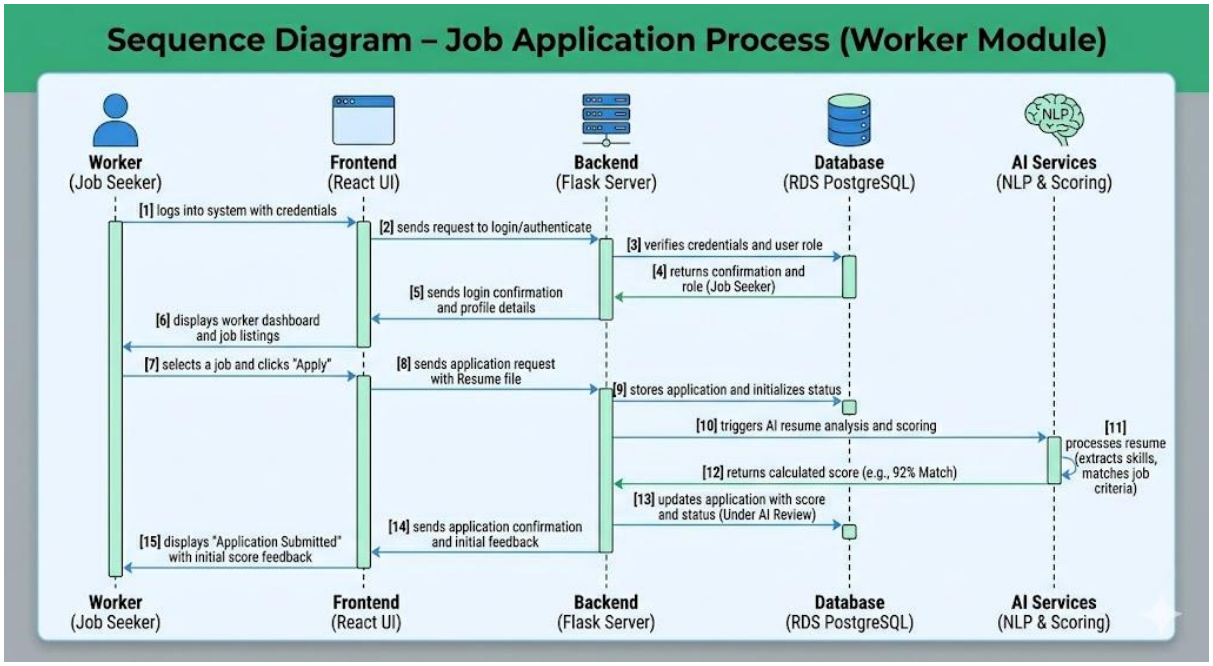



**Figure 4-2:** Sequence Diagram illustrating the job application process in the GetHired system.

### 4.1.3 Activity Diagram

An Activity Diagram in UML is used to represent the dynamic flow of activities within a system. It illustrates how different operations are performed step-by-step and how control flows from one activity to another.

In the GetHired system, the activity diagram helps in visualizing the workflow of user actions such as job searching, applying for jobs, and processing applications.

Activity diagrams support:

Sequential flow of actions

Decision-making (conditions)

Parallel processes (if any)

Start and end points of workflows

Activity Diagram – Job Application Workflow

The following describes the activity flow for a worker applying for a job in the GetHired system:

Start

User selects Login / Sign Up

System validates user credentials

If invalid → Redirect to login

If valid → Proceed to dashboard

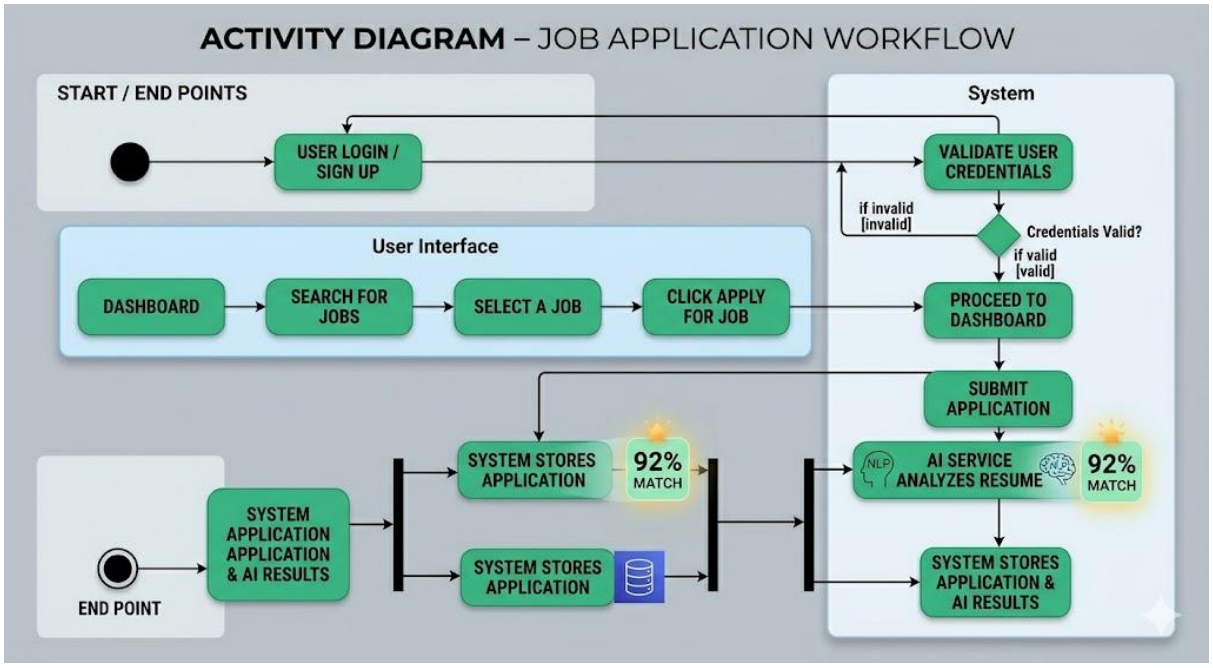
User searches for jobs

User selects a job

User clicks Apply for Job

System submits application

AI service analyzes resume and generates score




System stores application and AI results**Figure 4-3:** Activity Diagram representing the job application workflow in the GetHired system.

### 4.1.4 Class Diagram

In software engineering, a **Class Diagram** in UML is a type of static structure diagram that represents the structure of a system. It shows the system's classes, their attributes, methods (operations), and the relationships between them.

The class diagram is a fundamental part of object-oriented modeling and is used to:

- Represent system entities

- Define relationships between components
- Provide a blueprint for implementation
- Support database and backend design

In the GetHired system, the class diagram represents key entities such as users, jobs, applications, and AI analysis.

**Main Classes in GetHired System**

**1. User Class**

**Attributes:**

- user_id
- name
- email
- password
- role

**Methods:**

- register()
- login()
- updateProfile()

**2. Job Class**

**Attributes:**

- job_id
- title
- description
- location
- salary

- employer_id

**Methods:**

- createJob()
- updateJob()
- deleteJob()

**3. Application Class**

**Attributes:**

- application_id
- job_id
- user_id
- status
- applied_at

**Methods:**

- applyJob()
- updateStatus()

**4. AI_Analysis Class**

**Attributes:**

- analysis_id
- application_id
- score
- feedback

**Methods:**

- analyzeResume()
- generateFeedback()

**Relationships Between Classes**

- A **User (Employer)** can create multiple **Jobs** (1:N relationship)

- A **User (Worker)** can apply to multiple **Jobs** through **Applications** (1:N)
- A **Job** can have multiple **Applications** (1:N)
- Each **Application** can have one associated **AI Analysis** (1:1)

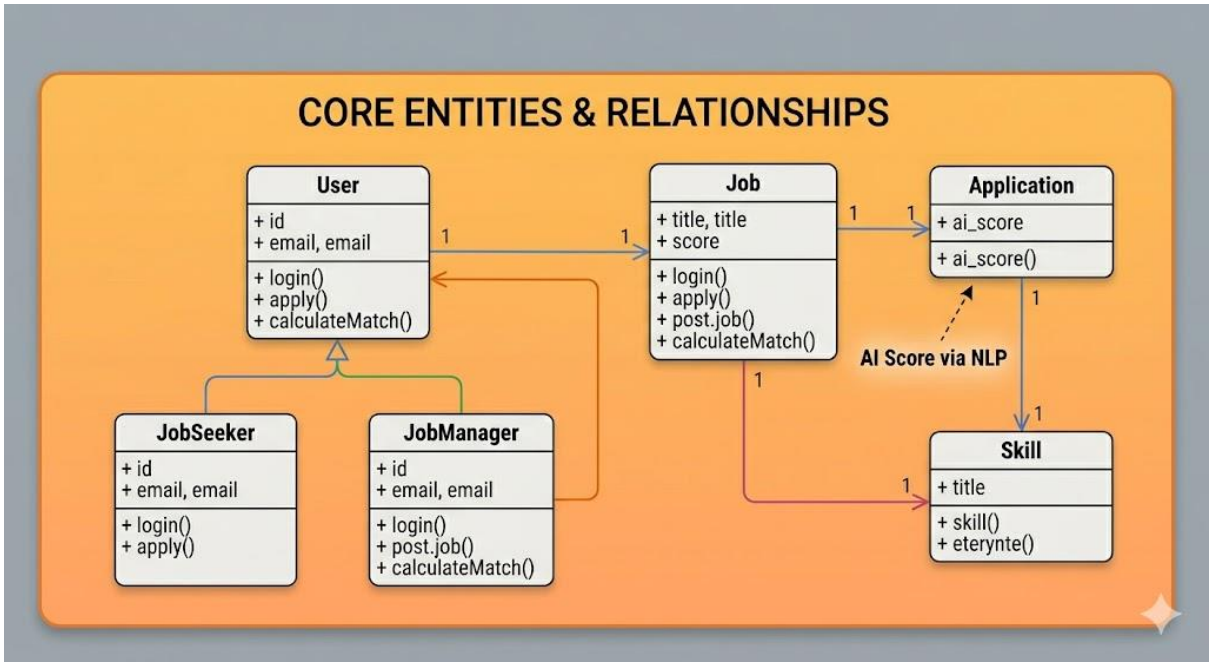



**Figure 4-4:** Class Diagram representing core entities and relationships in the GetHired system.

### 4.1.5 Deployment Diagram

A Deployment Diagram in UML represents the physical architecture of a system. It shows how software components are deployed on hardware or cloud infrastructure and how different nodes communicate with each other.

In the GetHired system, the deployment diagram illustrates how the application is hosted, how users access it, and how different services interact in a distributed environment.

Components of Deployment

The GetHired system consists of the following deployment nodes:

1. Client Devices (Users)

Web browsers (Desktop/Laptop)

Mobile browsers (Smartphones/Tablets)

Users interact with the system through:

React-based frontend

HTTPS requests

2. Frontend Server (Static Hosting)

Hosts the React application

Can be deployed using:

AWS S3 + CloudFront

Vercel / Netlify

Responsibilities:

Deliver UI to users

Handle client-side rendering

Communicate with backend APIs

3. Backend Server (Application Server)

Built using Flask (Python)

Hosted on:

AWS EC2 / Render / Heroku

Responsibilities:

Process API requests

Handle business logic

Manage authentication (JWT)

Communicate with database and AI services

4. Database Server

PostgreSQL (Production)

SQLite (Development)

Responsibilities:

Store users, jobs, applications

Maintain data consistency

Handle queries efficiently

5. External Services

AI APIs (Gemini / Groq) → Resume analysis, job matching

Email Service (SMTP) → Notifications and alerts

Cloud Storage → Resume uploads and files

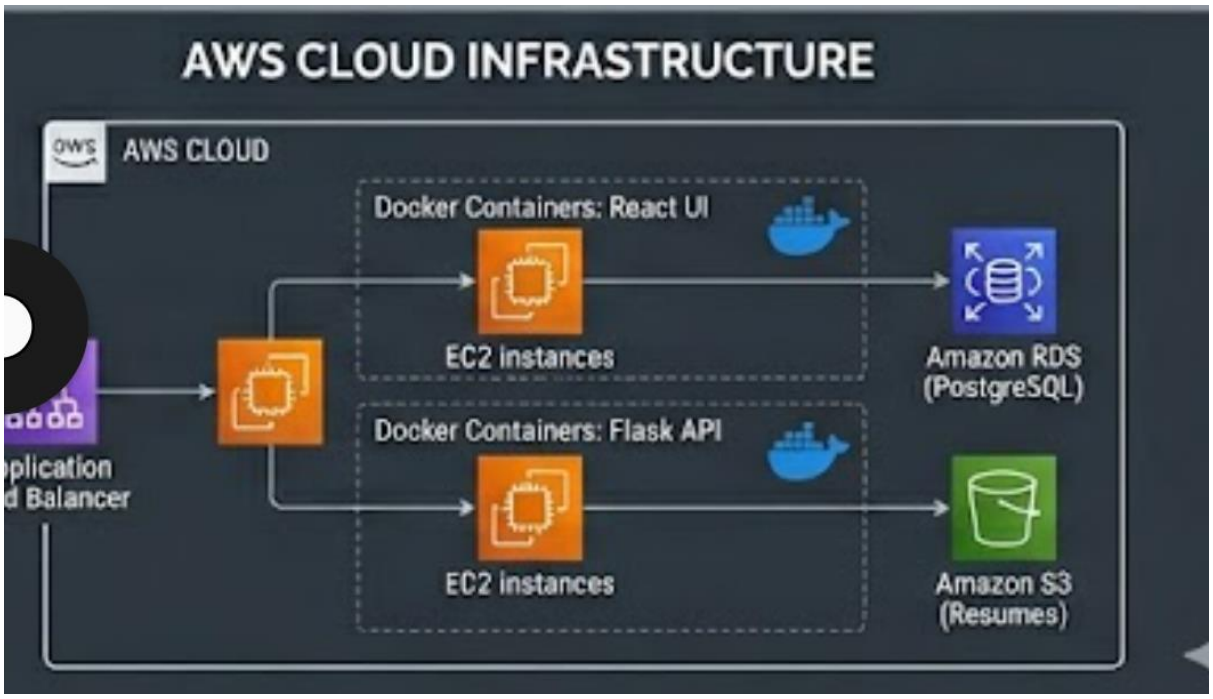



**Figure 4-5:** Deployment Diagram illustrating the infrastructure of the GetHired system across client devices, servers, and external services.

### 4.1.6 ER Diagram

An **Entity-Relationship (ER) Diagram** is a conceptual representation of the database structure. It illustrates how different entities (tables) are related to each other and how data flows within the system.

In the GetHired system, the ER diagram is used to model the relationships between core entities such as users, jobs, applications, and AI analysis data.

The ER model helps in:

- Designing the database schema

- Defining relationships between entities
- Ensuring data integrity and normalization
- Providing a clear visual understanding of system data

**Main Entities in GetHired System**

**1. Users**

Represents all users in the system, including workers, employers, and administrators.

**Attributes:**

- user_id (Primary Key)
- name
- email
- password
- role

**2. Jobs**

Represents job postings created by employers.

**Attributes:**

- job_id (Primary Key)
- title
- description
- location
- salary
- employer_id (Foreign Key → Users)

**3. Applications**

Represents job applications submitted by workers.

**Attributes:**

- application_id (Primary Key)
- job_id (Foreign Key → Jobs)

- user_id (Foreign Key → Users)
- status
- applied_at

**4. AI_Analysis**

Represents AI-generated analysis data.

**Attributes:**

- analysis_id (Primary Key)
- application_id (Foreign Key → Applications)
- score
- feedback

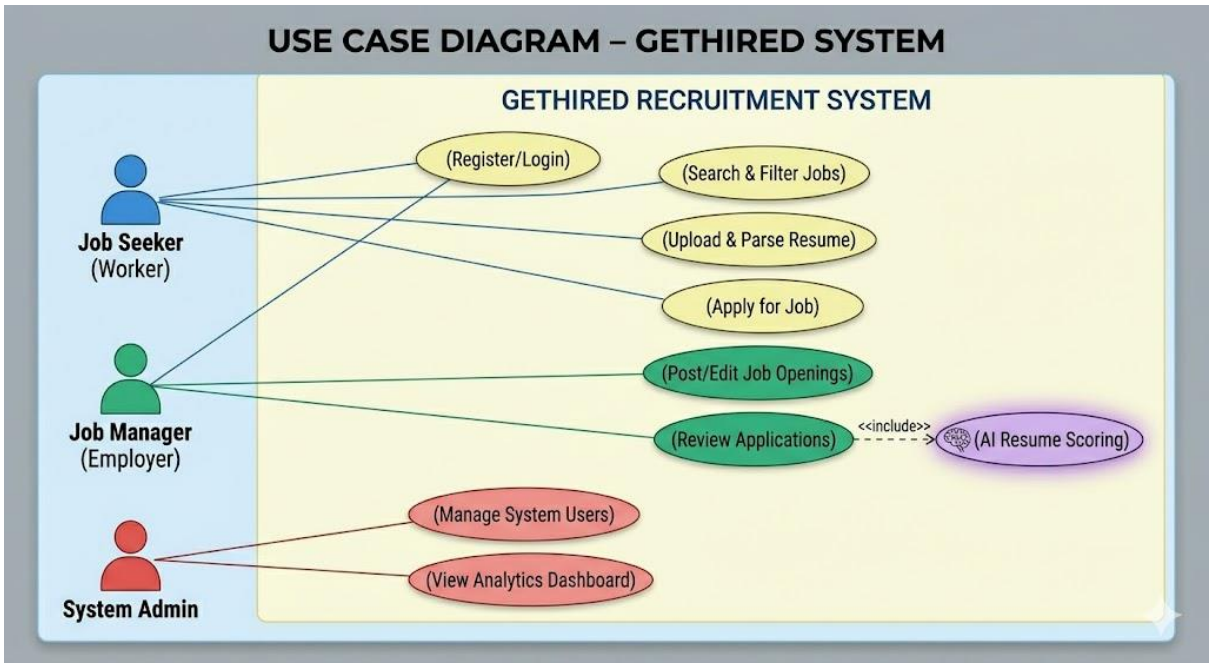


**Figure 4-6:** ER Diagram representing relationships between Users, Jobs, Applications, and AI Analysis in the GetHired system.

# CHAPTER - 5

# IMPLEMENTATION

# 5. IMPLEMENTATION

## 5.1 Sample Code

The implementation of the GetHired system is carried out using a combination of **Flask (Backend)** and **React (Frontend)**. The backend handles API requests and business logic, while the frontend provides an interactive user interface.

### 5.1.1 Code for Home Page (Frontend – React)

The following code represents a basic home page of the GetHired application where users can navigate to login or register.

```
export default function App() {
  return (
    <AuthProvider>
      <TransitionProvider>
        <PageTransition>
          <Routes>
            <Route path="/" element={<HomePage />} />
            <Route path="/auth" element={<AuthPage />} />

            <Route element={<ProtectedRoute />}>
              <Route path="/app" element={<DashboardLayout />}>
                <Route index element={<DashboardIndexRedirect />} />

                <Route element={<RoleRoute allowedRoles={['worker']} />}>
                  <Route path="worker" element={<WorkerDashboard />} />
                  <Route path="worker/profile" element={<WorkerProfilePage />} />
                  <Route path="worker/jobs" element={<WorkerJobsPage />} />
                  <Route path="worker/jobs/:jobId" element={<WorkerJobDetailPage />} />
                  <Route path="worker/saved" element={<WorkerSavedJobsPage />} />
                  <Route path="worker/applications" element={<WorkerApplicationsPage />} />
                </Route>

                <Route element={<RoleRoute allowedRoles={['employer']} />}>
                  <Route path="employer" element={<EmployerDashboard />} />
                  <Route path="employer/post-job" element={<EmployerPostJobPage />} />
                  <Route path="employer/jobs" element={<EmployerManageJobsPage />} />
                  <Route path="employer/jobs/:jobId" element={<EmployerJobDetailPage />} />
                  <Route path="employer/profile" element={<EmployerProfilePage />} />
                </Route>
```

### 5.1.2 Backend API Example (Flask – Authentication)

The following code demonstrates a simple login API using Flask:

```
from flask import Flask, request, jsonify
from werkzeug.security import check_password_hash
import jwt
import datetime

app = Flask(__name__)
app.config['SECRET_KEY'] = 'your_secret_key'

# Example user (for demo)
users = [
    {"email": "test@example.com", "password": "hashed_password"}
]

@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    email = data.get("email")
    password = data.get("password")

    user = next((u for u in users if u["email"] == email), None)

    if user and check_password_hash(user["password"], password):
        token = jwt.encode({
            'email': email,
            'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=2)
```

5.1.3 Job API Example:

```
@app.route('/jobs', methods=['GET'])
def get_jobs():
    jobs = [
        {"id": 1, "title": "Software Engineer"},
        {"id": 2, "title": "Data Analyst"}
    ]
    return jsonify(jobs)
```

5.1.4 User Registration API (Flask)

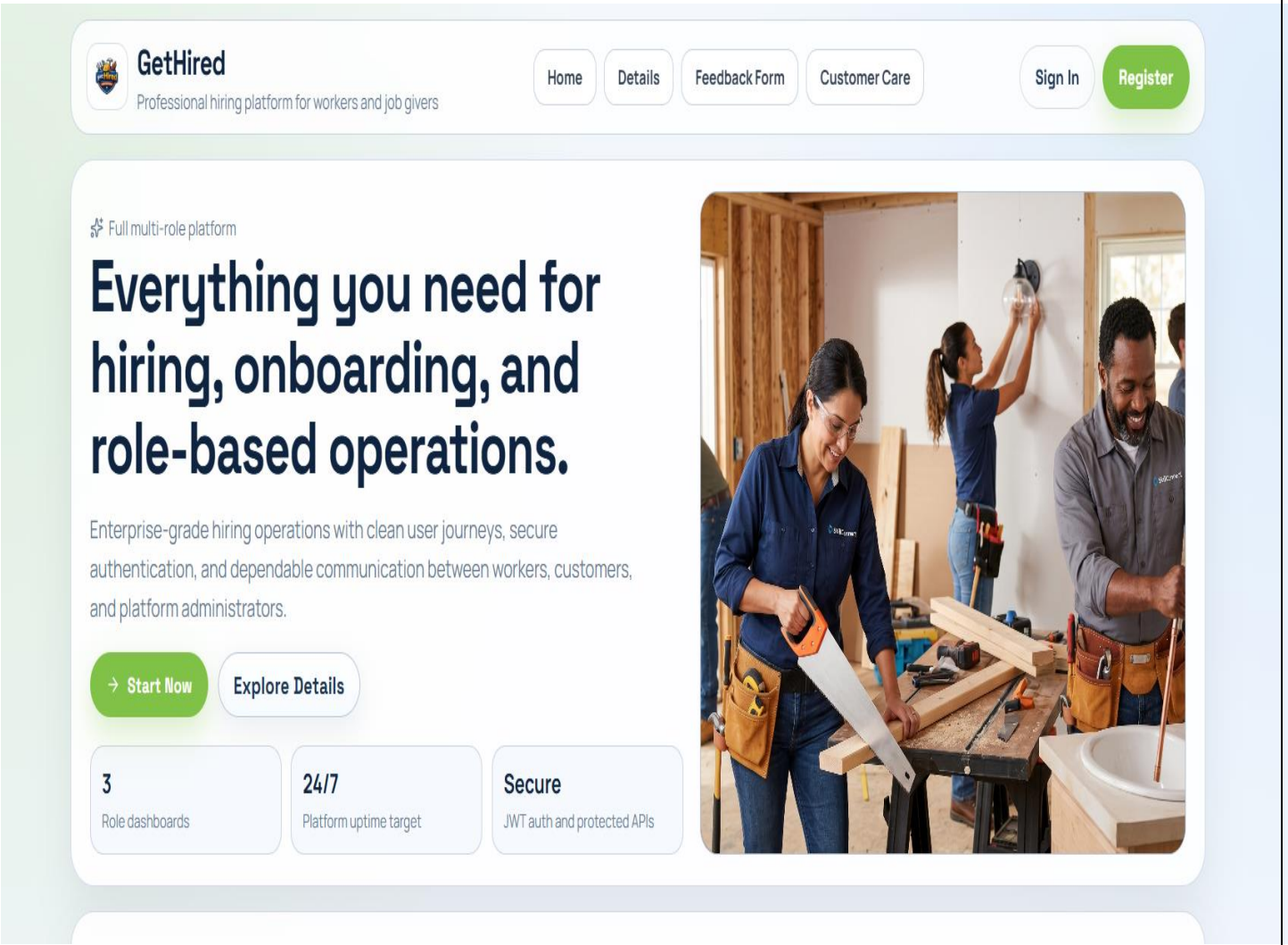
```
@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()

    new_user = {
        "name": data.get("name"),
        "email": data.get("email"),
        "password": data.get("password"),
        "role": data.get("role", "worker")
    }
```

5.1.7 React Job Listing Page

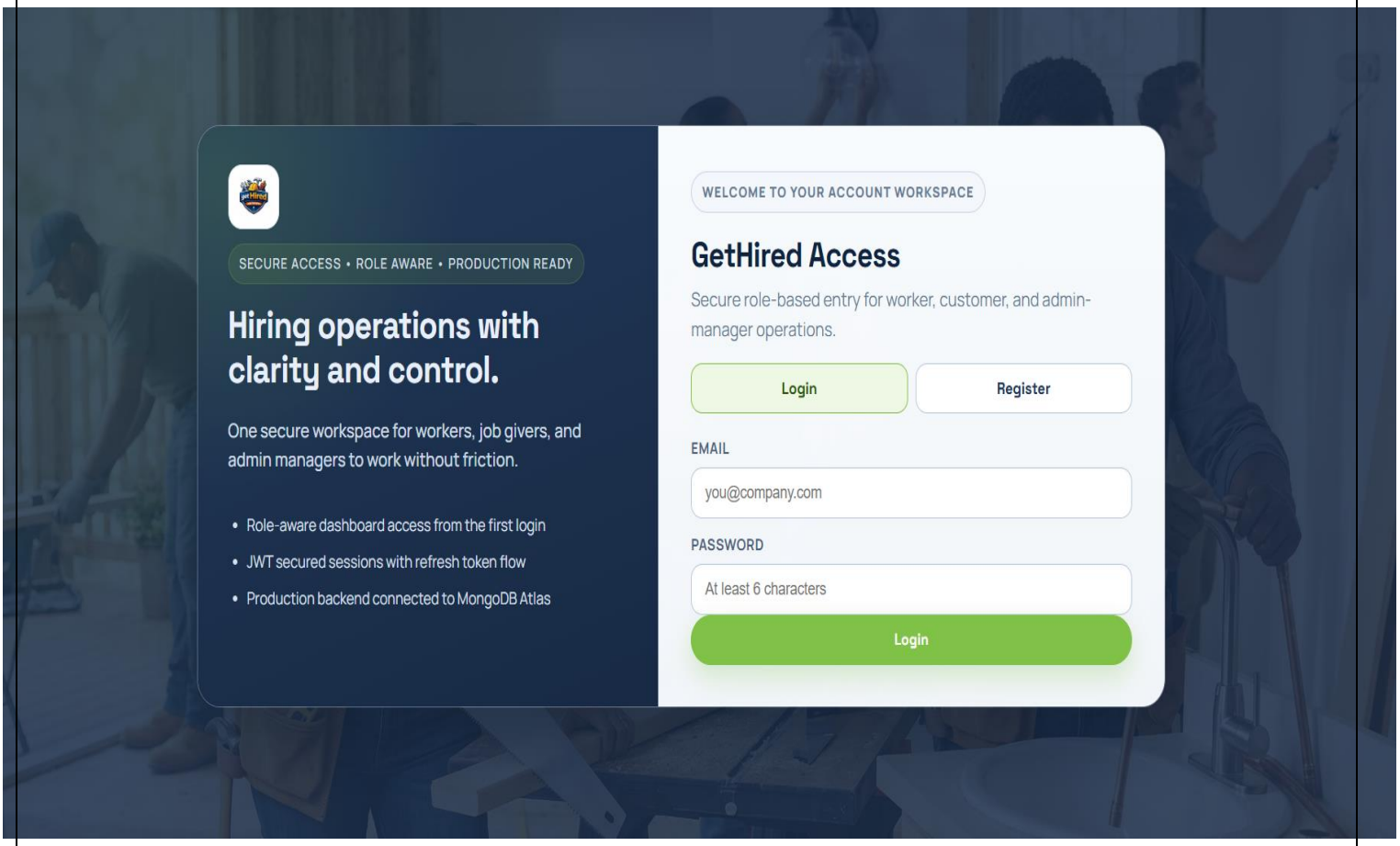
```
import React, { useEffect, useState } from "react";
import axios from "axios";

const Jobs = () => {
  const [jobs, setJobs] = useState([]);

  useEffect(() => {
    axios.get("http://localhost:5000/jobs")
      .then(res => setJobs(res.data))
      .catch(err => console.log(err));
  }, []);

  return (
    <div>
      <h2>Available Jobs</h2>
      {jobs.map(job => (
        <div key={job.id}>
          <h3>{job.title}</h3>
        </div>
      ))}
    </div>
```

## 5.2 Screen Captures

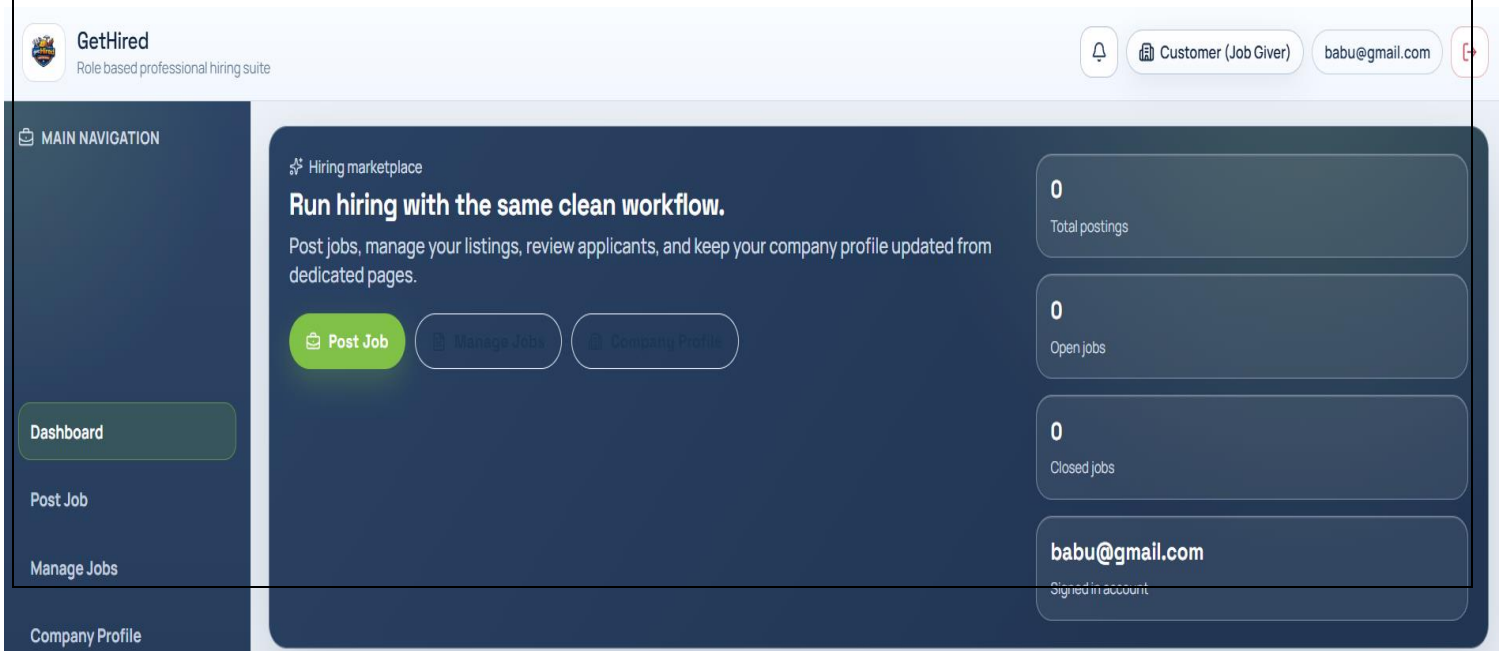
### 5.2.1 home screen:



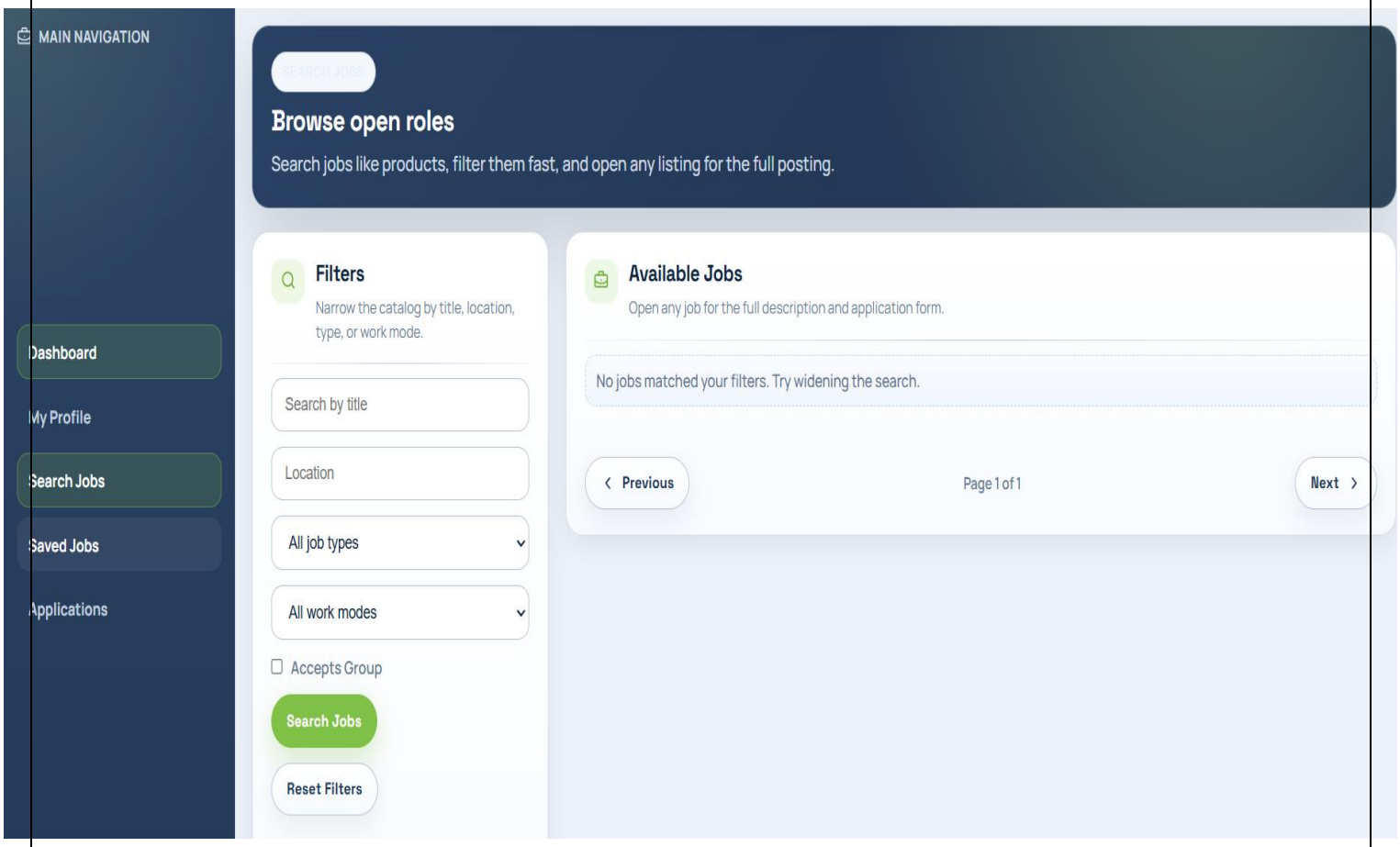
5.2.2 Login Screen




5.2.3 Dashboard Screen of Job Giver



5.2.4 Dashboard Screen of Job seeker;



# CHAPTER - 6

# TESTING

# 6. TESTING

## 6.1 Software Testing

Software testing is the process of validating and verifying that the GetHired application meets the specified requirements and functions correctly. It helps identify bugs, errors, and performance issues before deployment.

Testing ensures:

System reliability and stability

Correct functionality of features

Security and performance standards

Better user experience

Various testing methods are applied to the GetHired system, including manual testing, black box testing, white box testing, performance testing, and load testing.

## 6.2 Black Box Testing

Black box testing evaluates the system without knowledge of internal code structure. It focuses on input and output behavior.

In the GetHired system, black box testing is applied to:

User registration and login

Job search and filtering

Job application submission

API responses

Example:

Input: Valid login credentials

Output: Successful authentication and dashboard access

6.3 White Box Testing

White box testing involves testing the internal logic, code structure, and data flow of the system.

In GetHired, it includes:

API endpoint logic verification

Database query validation

Authentication and authorization logic

AI processing functions

This ensures that all internal operations are functioning correctly.

6.4 Performance Testing

Performance testing evaluates how efficiently the system performs under normal and peak conditions.

For GetHired:

API response time is tested

Database query performance is analyzed

AI processing time (resume scoring) is evaluated

The system is optimized to ensure quick response times and smooth user experience.

6.5 Load Testing

Load testing checks how the system behaves under heavy usage.

In GetHired:

Multiple users accessing job listings simultaneously

High number of API requests

Concurrent job applications

The system is tested to ensure it can handle increased traffic without failure.

## 6.6 Manual Testing

Manual testing is performed by testers to verify system functionality without automation tools.

Test cases include:

User registration and login

Job posting by employers

Job application by workers

Viewing application status

Manual testing ensures that the application behaves as expected from a user perspective.

| **Test Case Name** | **Job Application Submission** |
|---|---|
| **Description** | Verify that a worker can successfully apply for a job when all required fields are provided |
| **Input** | Valid user ID, job ID, and application details |
| **Expected Output** | Application is successfully submitted and stored in the database |
| **Actual Output** | Application submitted successfully |
| **Status** | Passed |

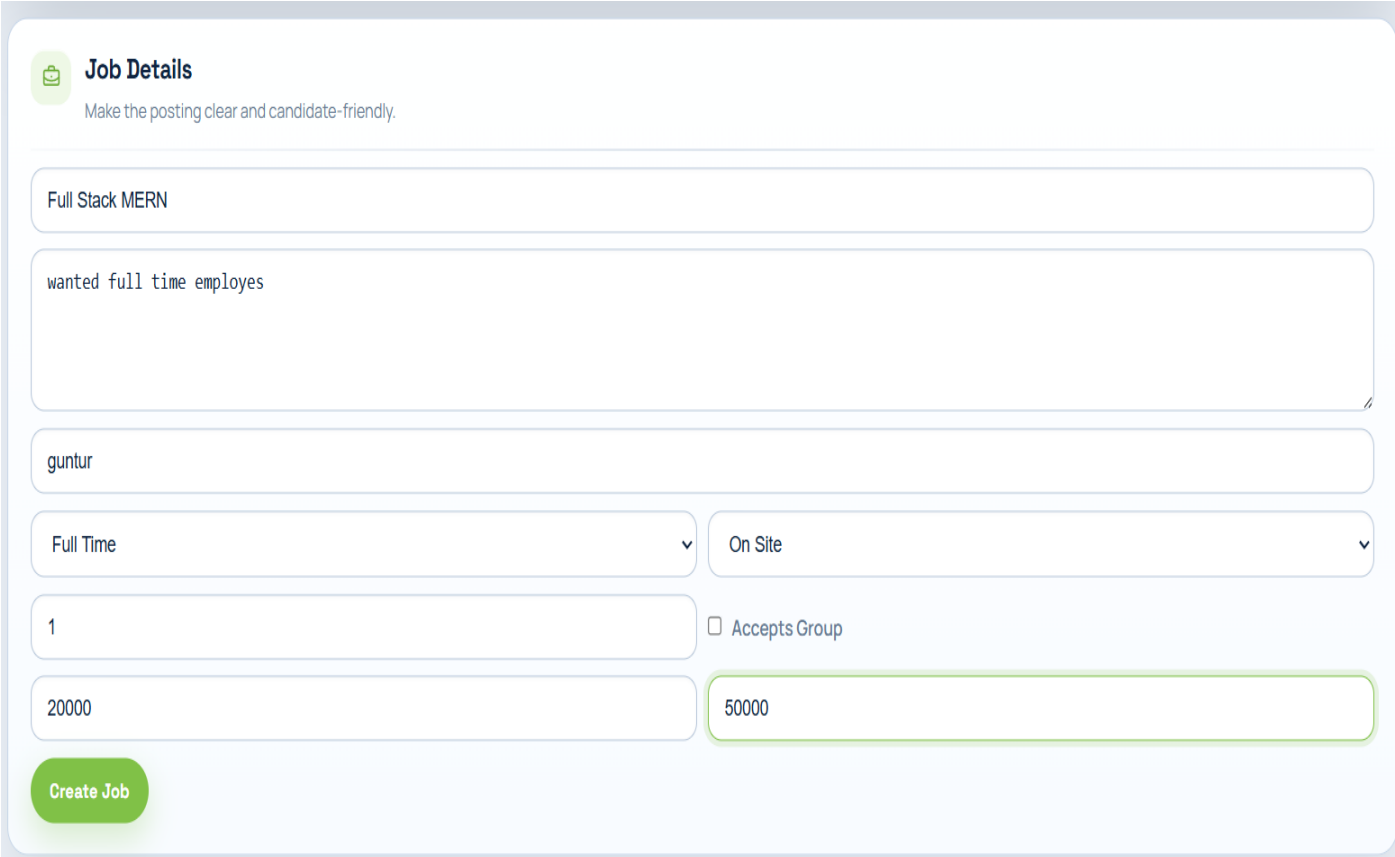



**Figure 6-1:** Test Case for Job Application Submission in the GetHired system.

| **Test Case Name** | **Updating Application Status** |
|---|---|
| **Description** | Verify that an employer can update the status of a job application |
| **Input** | Valid application ID and status (Accepted / Rejected / Reviewing) |
| **Expected Output** | Application status is successfully updated in the system |
| **Actual Output** | Application status updated successfully |
| **Status** | Passed |

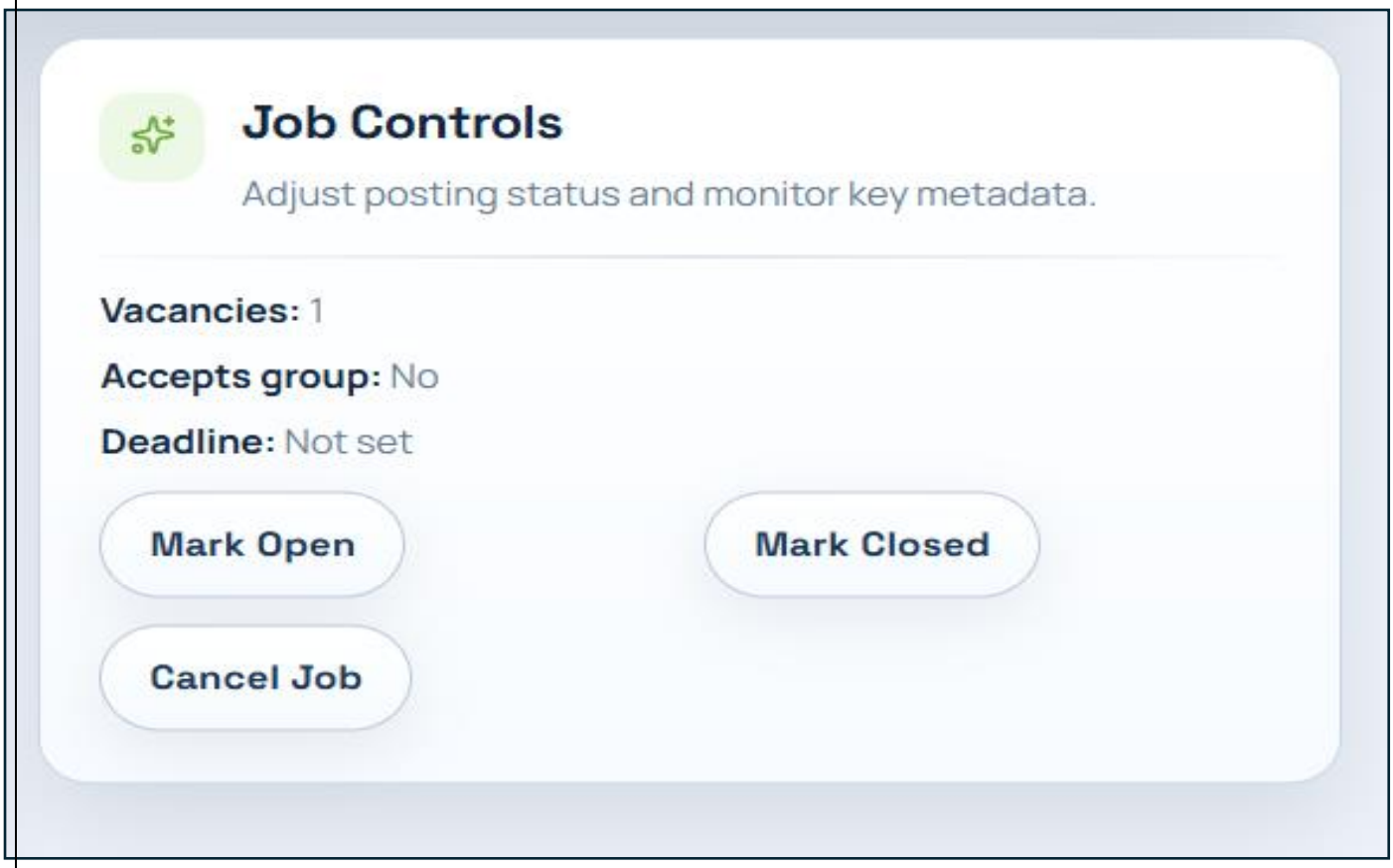



**Figure 6-2:** Test Case for Updating Application Status in the GetHired system.

# CHAPTER - 7

# RESULTS &CHALLENGES

# 7. RESULTS AND CHALLENGES

## 7.1 Results

The GetHired system successfully provides a **centralized recruitment platform** where job seekers and employers can interact efficiently. The platform simplifies the hiring process by enabling users to access job opportunities and manage applications in a streamlined manner.

The system achieves the following results:

- Users can register and log in securely
- Workers can search for jobs based on various filters
- Workers can apply for jobs from anywhere
- Employers can post and manage job listings
- Employers can review applications and update their status
- AI features provide resume scoring and job matching insights

The system supports two primary types of users:

**1. Employer**

- Employers can register and log in to the platform
- They can create, update, and delete job postings
- Employers can view applications submitted by workers
- They can accept, reject, or review applications
- AI-based evaluation helps in selecting suitable candidates

**2. Worker (Job Seeker)**

- Workers can register and log in

- They can search for jobs based on preferences
- Workers can apply for jobs easily
- They can track the status of their applications
- They receive AI-based feedback and recommendations

Overall, the system improves:

- Recruitment efficiency
- Decision-making using AI
- User experience through a modern interface

### 7.2 Challenges

During the development of the GetHired system, several challenges were encountered:

- **Understanding-Requirements:**
  Translating real-world recruitment needs into system features required careful analysis.
- **Frontend–Backend:**
  Ensuring smooth communication between React frontend and Flask backend APIs was challenging.
- **Database-Design:**
  Structuring relational data (users, jobs, applications) while maintaining scalability required proper planning.
- **AI-Integration:**
  Integrating AI services for resume analysis and ensuring accurate results required experimentation and tuning.
- **Authentication&Security:**
  Implementing secure authentication (JWT) and protecting APIs was critical.
- **Performance:**
  Ensuring fast API responses and handling multiple users efficiently required optimization.
- **User-Design:**
  Creating a responsive and user-friendly interface for different devices required careful design.

# CHAPTER – 8

# CONCLUSIONS & FUTURE WORK

# 8. CONCLUSION

## 8.1 Conclusion

The GetHired system has been successfully designed and developed to meet the requirements of a modern recruitment platform. The project demonstrates how technology can simplify and improve the hiring process by connecting job seekers and employers through a centralized system.

This project provided valuable experience in:

- Full-stack development (React + Flask)
- API design and integration
- Database modeling
- AI feature integration

The development process involved multiple challenges, but overcoming them enhanced both technical and problem-solving skills. The final system is functional, scalable, and capable of handling real-world recruitment scenarios.

## 8.2 Scope for Future Work

The GetHired system can be further enhanced with additional features and improvements:

- Integration of real-time notifications (email/SMS alerts)
- Advanced AI features such as interview prediction and skill gap analysis
- Implementation of real-time chat between employers and workers
- Mobile application development (Android/iOS)
- Integration with third-party platforms like LinkedIn or job portals
- Improved recommendation system using machine learning models
- Deployment with auto-scaling and load balancing

### 8.3 Limitations

Despite its successful implementation, the system has some limitations:

- AI features are currently basic and can be improved with more advanced models
- Limited real-time communication features
- Performance may vary under very high user load without scaling

## BIBLIOGRAPHY


[1] Flask Documentation – Flask Web Framework

Available at: https://flask.palletsprojects.com/

[2] React Documentation – React JavaScript Library

Available at: https://react.dev/

[3] PostgreSQL Documentation – Open Source Database

Available at: https://www.postgresql.org/

[4] REST API Design – Best Practices Guide

Available at: https://restfulapi.net/

[5] JWT Authentication – JSON Web Tokens Introduction

Available at: https://jwt.io/introduction/

[6] Python Documentation – Official Python Docs

Available at: https://docs.python.org/3/

[7] Git Documentation – Version Control System

Available at: https://git-scm.com/docs

[8] Google AI / Gemini API Documentation

Available at: https://ai.google.dev/